LLM | Large Language Models | Python | NLP | Transformers

Building a Large Language Model (LLM) from Scratch

By Abdul Rauf Jatoi



Abdul Rauf Follow 9 min read · Dec 18, 2024

66 2 1

Hi there, I'm Rauf, and today I'm going to walk you through the fascinating process of creating a Large Language Model (LLM) from scratch. LLMs like GPT are powerful AI models capable of understanding and generating human-like text. But have you ever wondered how they work behind the scenes? Let's dive in and explore the entire process, step by step 😊



Title

LLMs are based on deep learning techniques, utilizing architectures like transformers to process large volumes of text data. They are built through a series of essential steps: **data collection, preprocessing, model architecture, training, fine-tuning, and deployment.**

What is an LLM?

LLM stands for **Large Language Model.** It is a type of advanced artificial intelligence model trained on large amounts of text data to understand and generate human-like text. These models are built using deep learning techniques and can perform a variety of natural language processing (NLP) tasks.

Components of an LLM

Embedding Layers

- Converts words, phrases, or tokens into numerical vectors.
- These vectors capture the semantic meaning of the text, helping the model understand relationships between words.

Attention Mechanism

- Helps the model focus on the most relevant parts of the input text.
- Example: If the task is translation, the model "attends" to the words in the input sentence that are most relevant to the word being generated.

Transformer Blocks

- The core building blocks of an LLM.
- Consist of two main components:
- **Self-Attention Layers:** Process relationships between words in the input text.
- **Feedforward Neural Networks:** Perform additional computations to refine the understanding of the text.
- Multiple transformer blocks are stacked to improve the model's performance on complex tasks.

Positional Encoding

- Since transformers don't process sequences in order, positional encoding helps the model understand the order of words in a sentence.

Output Layers

- Generates predictions based on the processed input.
- For tasks like text generation, the output is a sequence of words or tokens.

Tasks LLMs Can Perform

Text Generation

- Create human-like text based on a given prompt.
- Example: Writing essays, stories, or code snippets.

Summarization

- Generate concise summaries of longer texts.
- Example: Summarizing news articles or research papers.

Translation

- Translate text from one language to another.
- Example: English to French.

Question Answering

- Provide answers to questions based on input text or general knowledge.
- Example: Answering trivia or explaining concepts.

Sentiment Analysis

- Determine the emotional tone of text (positive, negative, or neutral).
- Example: Analyzing customer reviews.

Text Completion

- Predict and complete unfinished sentences or paragraphs.
- Example: Auto-completing emails or search queries.

Conversational AI

- Power chatbots and virtual assistants to have meaningful conversations.
- Example: Customer support or personal assistants like Siri.

Code Generation

- Generate or debug code for programming tasks.
- Example: Writing Python scripts.

Steps of Creating an LLM



step 1

Step 1: Data Collection

Why is Data Collection Important?

- **Large, diverse, and clean datasets** are essential for training a robust model.
- **Large**: To cover a wide variety of use cases and scenarios.
- **Diverse**: To include different topics, languages, and styles of writing.
- **Clean**: To ensure the data is accurate, relevant, and free from noise or errors, which improves the model's performance.

Ways to Collect Text Data

Open Datasets

- Use publicly available datasets such as:

- **Common Crawl**: Large-scale web data.
- **Wikipedia**: Well-structured and high-quality text.
- Tools: Download directly or use APIs provided by these platforms.

APIs

- Access text data programmatically through APIs:
- **Reddit API**: For user discussions.
- **Twitter API**: For tweets.
- Benefit: APIs provide structured and updated text data.

Web Scraping

- Scrape websites to extract text using Python libraries like `BeautifulSoup`.
- Use it for data that is not available through APIs but is publicly accessible.

Code Example: Scraping Text Data from a Website

Here's how to scrape text from a website using Python:

```
import requests
from bs4 import BeautifulSoup

# Step 1: Specify the URL
url = "https://wikipedia.com"

# Step 2: Send a GET request to the website
response = requests.get(url)

# Step 3: Parse the website content
soup = BeautifulSoup(response.text, "html.parser")
```

```
# Step 4: Extract all text from the page
text_data = soup.get_text()

# Step 5: Print the first 500 characters of the text
print(text_data[:500])
```

Steps in the Code:

1. **`requests.get(url)`** : Fetches the website's content.
2. **`BeautifulSoup(response.text, "html.parser")`** : Parses the HTML content.
3. **`soup.get_text()`** : Extracts the plain text from the HTML.
4. **`print(text_data[:500])`** : Displays the first 500 characters for a quick preview.

Things to Keep in Mind

- **Respect Website Policies**: Check the site's terms of use or robots.txt file to ensure scraping is allowed.
- **Avoid Overloading Servers**: Use rate limits and polite scraping practices.
- **Clean the Data**: Remove unwanted tags, special characters, or advertisements.



step 2

Step 2: Data Preprocessing

Why is Data Preprocessing Important?

Open in app ↗ Sign up Sign in
Medium Search Write

- **Preprocessing** ensures that the data is clean and standardized, improving the model's understanding and performance.

Common Preprocessing Tasks

Removing Stop Words

- Stop words (e.g., "the," "is") don't add much value to the model and can be removed.

Tokenization

- Break text into smaller units like words or sentences.

Lowercasing

- Convert all text to lowercase for uniformity.

Handling Special Characters

- Remove symbols, numbers, or unwanted characters that don't contribute to the task.

Libraries for Preprocessing

- **`nltk`** : Natural Language Toolkit for tokenization, stop word removal, etc.
- **`spaCy`** : Advanced NLP processing.
- **`pandas`** : Data manipulation for handling text in large datasets.

Code Example: Preprocessing Text with NLTK

Here's a simple example using `nltk` to preprocess text:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Download required NLTK data
nltk.download('punkt')
nltk.download('stopwords')

# Sample text
text = "My name is Abdul Rauf jatoi"

# Step 1: Tokenize text into words
tokens = word_tokenize(text)
```

```
# Step 2: Convert to lowercase and remove non-alphanumeric tokens
tokens = [word.lower() for word in tokens if word.isalnum()]

# Step 3: Remove stop words
filtered_tokens = [word for word in tokens if word not in stopwords.words('engli

# Display the preprocessed tokens
print(filtered_tokens)
```

Steps in the Code:

1. **`word_tokenize(text)`** : Splits text into individual words.
2. **`[word.lower() for word in tokens if word.isalnum()]`** : Converts words to lowercase and removes non-alphanumeric tokens.
3. **`stopwords.words('english')`** : Filters out common stop words.

Output

For the given example text, the output will be:

```
['My', 'name', 'is', 'Abdul', 'Rauf', 'jatoi']
```

Key Notes

- **Efficiency**: Preprocessing makes the data simpler for the model to process.
- **Flexibility**: You can customize the steps based on your specific task.

- **Tools**: NLTK is great for beginners; spaCy is better for more advanced tasks.



step 3

Step 3: Model Architecture and Training

What is a Transformer Model?

- **Transformers** are advanced models designed for sequence-to-sequence tasks.
- Key components of a transformer include:

1. **Attention Mechanism**: Helps the model focus on relevant parts of the input sequence.
2. **Self-Attention**: Allows each token to attend to every other token in the sequence.

3. **Feedforward Layers:** Applies transformations to each token independently after attention.
4. **Layer Normalization:** Stabilizes training and improves convergence.

Libraries

- **TensorFlow:** Offers built-in layers for transformers.
- **PyTorch:** Flexible framework for implementing custom architectures.

Code Example: Define a Transformer in TensorFlow

Here's a simple example to create a transformer block and model:

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, LayerNormalization, MultiHeadA

# Define a single transformer block
def transformer_block(input, num_heads, key_dim):
    # Step 1: Multi-head attention
    attention = MultiHeadAttention(num_heads=num_heads, key_dim=key_dim)(input,
    # Step 2: Add & Normalize
    attention = LayerNormalization()(attention + input)
    # Step 3: Feedforward network
    dense = Dense(128, activation='relu')(attention)
    # Step 4: Add & Normalize
    output = LayerNormalization()(dense + attention)
    return output

# Input layer
input_layer = Input(shape=(None, 128))  # Sequence length is variable, feature s
# Transformer block
transformer_output = transformer_block(input_layer, num_heads=8, key_dim=64)
# Model definition
model = tf.keras.Model(inputs=input_layer, outputs=transformer_output)

# Model summary
model.summary()
```

Steps in the Code:

Input Layer: Accepts a sequence of features (`shape=(None, 128)`).

Get Abdul Rauf's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

Attention Mechanism:

- `MultiHeadAttention` computes self-attention for multiple heads.

Add & Normalize:

- Residual connection added to attention output and normalized with `LayerNormalization`.

Feedforward Layer:

- Dense layer with ReLU activation processes each token independently.

Final Output:

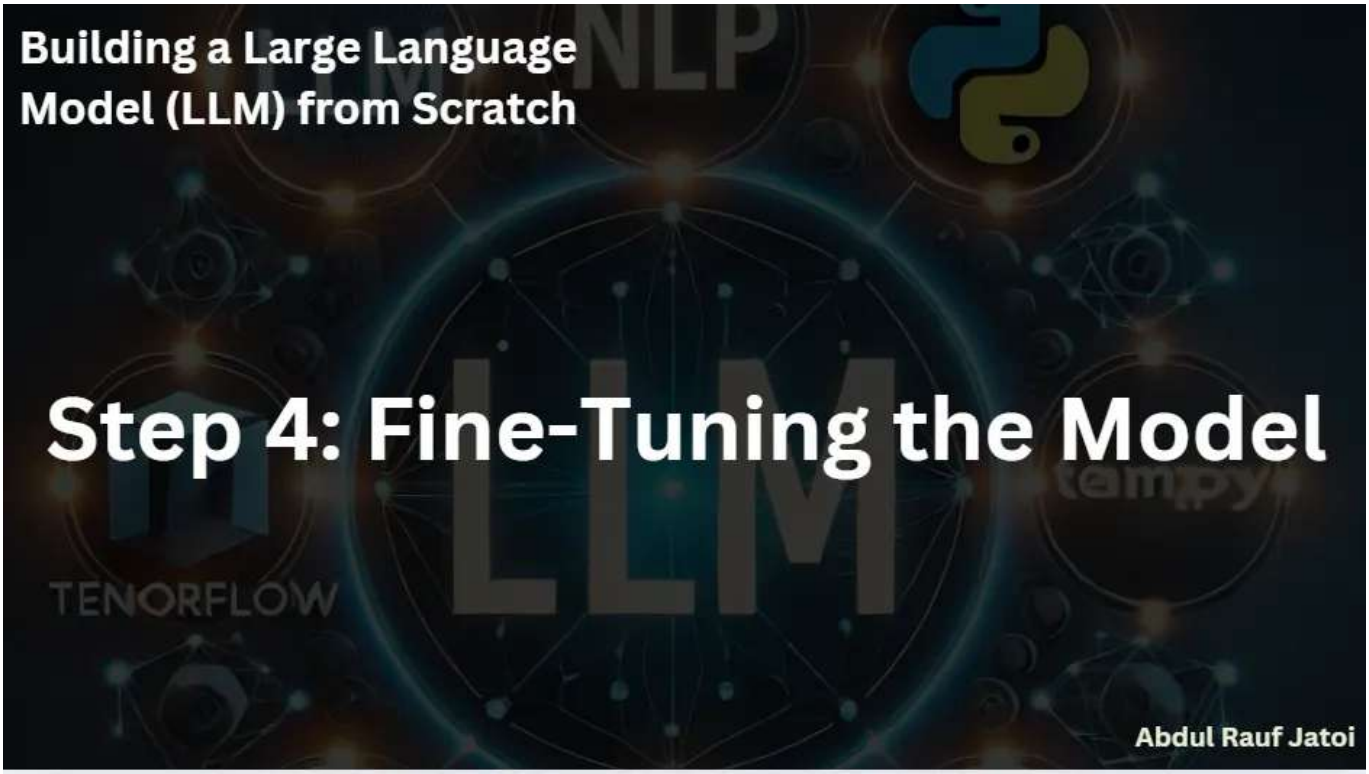
- Residual connection added again, followed by normalization.

Model Summary

When you run this code, you'll get a summary showing the layers and parameters in your transformer block.

Scaling Up

- This basic block can be stacked to create a full transformer model.
- Use additional components like positional encodings for larger-scale transformer architectures.



step 4

Step 4: Fine-Tuning the Model

What is Fine-Tuning?

- Fine-tuning involves adapting a pre-trained model (like GPT) to a specific task, such as text generation, summarization, or classification.
- **Benefits:**
- Saves time and resources by leveraging pre-trained knowledge.
- Achieves high performance on specific tasks with minimal labeled data.

Libraries for Fine-Tuning

- **Hugging Face Transformers:** Provides tools for working with pre-trained models and datasets.
- **Datasets:** A dataset specific to the task is required for fine-tuning.

Code Example: Fine-Tuning GPT

Here's a simple example using Hugging Face Transformers to fine-tune GPT:

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
from transformers import Trainer, TrainingArguments
import torch

# Step 1: Load the tokenizer and model
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2')

# Step 2: Prepare the dataset (replace `my_dataset` with your dataset)
# Ensure `my_dataset` is a PyTorch dataset with tokenized input
# Example tokenization
texts = ["Hello, how are you?", "Fine-tuning is fun!"]
encodings = tokenizer(texts, truncation=True, padding=True, return_tensors="pt")
labels = encodings.input_ids.clone()

my_dataset = torch.utils.data.TensorDataset(encodings.input_ids, labels)

# Step 3: Define training arguments
train_args = TrainingArguments(
    output_dir='./results',          # Directory to save the model
    per_device_train_batch_size=4, # Batch size
    num_train_epochs=1,              # Number of epochs
    save_steps=10_000,               # Steps to save checkpoints
    save_total_limit=2,              # Maximum number of saved checkpoints
    logging_dir='./logs',            # Directory for logs
    logging_steps=500,               # Log every 500 steps
)

# Step 4: Create Trainer and train the model
trainer = Trainer(
    model=model,
    args=train_args,
```

```
    train_dataset=dataset,
)

trainer.train()
```

Steps in the Code:

Load Pre-trained Model:

- `GPT2Tokenizer` and `GPT2LMHeadModel` load GPT-2's tokenizer and model.

Prepare Dataset:

- Tokenize your dataset using the tokenizer.
- Create a PyTorch dataset from tokenized input and labels.

Define Training Arguments:

- Customize parameters like batch size, epochs, and saving intervals.

Train the Model:

- Use `Trainer` for fine-tuning.

Output

The model is fine-tuned on your dataset and saved in the specified directory (`./results`). It can now be used for your task (e.g., text generation or specific applications).

Note: Replace `dataset` with a suitable dataset relevant to your task.



step 5

Step 5: Deploying the Model

Why Deploy a Model?

- Deployment makes your model accessible for real-world applications through APIs or web interfaces.

Deployment Options:

1. **Vercel**: Easy-to-use platform for deploying apps with minimal configuration.
2. **Hugging Face Spaces**: Free hosting for machine learning apps using Streamlit or Gradio.

3. **AWS**: Scalable cloud platform for deploying models with high availability.

Using flask

Flask is a lightweight framework for creating web APIs. Here's how to deploy a text generation model with Flask:

Code Example: Deploying with Flask

```
from flask import Flask, request, jsonify
from transformers import pipeline

# Initialize Flask app and model pipeline
app = Flask(__name__)
model = pipeline("text-generation", model="gpt2")

# Define API endpoint
@app.route("/generate", methods=["POST"])
def generate():
    # Get input text from the request
    input_text = request.json["input"]
    # Generate text using the model
    output = model(input_text, max_length=50)
    # Return the generated text as a JSON response
    return jsonify(output)

# Run the app
if __name__ == "__main__":
    app.run(debug=True)
```

Steps in the Code:

Import Libraries:

- `Flask` for web API.
- `pipeline` from Hugging Face for easy access to the model.

Initialize Model:

- Use the `pipeline` function to load the GPT-2 text generation model.

Create API Endpoint:

- `/generate` : Accepts POST requests with JSON input containing the text to process.
- Returns generated text as JSON output.

Run Flask App:

- Start the server locally with `debug=True` .

How to Test the API

1. Save the script as `app.py` .
2. Run the Flask app:

```
python app.py
```

3. Use tools like **Postman** or **cURL** to send POST requests:

- Example POST request:

```
{
  "input": "Once upon a time"
```

```
}
```

4. The server responds with generated text.

Deployment Beyond Localhost

- Use platforms like **Vercel** or **AWS Elastic Beanstalk** to host your Flask app.
- Containerize the app using Docker for scalability and easy deployment.

This approach makes your model accessible as a RESTful API!

Conclusion

In this Article, we covered the key steps to create and deploy an LLM:

1. **Data Collection**: Gather large, diverse, and clean datasets.
2. **Data Preprocessing**: Clean and format data for better training.
3. **Model Architecture and Training**: Build and train transformer models.
4. **Fine-Tuning**: Adapt pre-trained models for specific tasks.
5. **Deployment**: Make the model accessible through APIs or web apps.

Understanding each step is crucial for building effective LLMs. Experiment with the provided code and customize it for your needs.

Feedback Welcome!
I hope this guide helps you understand the process of creating an LLM. Let

me know your thoughts or questions in the comments below and thanks for reading 😊

LLM + | Large Language Models + | Python + | NLP +

Transformers +



Written by Abdul Rauf

120 followers · 8 following

I write about Artificial Intelligence (AI) , Machine Learning (ML) , Web Development and Tech stuffs. Sometimes my stories too (some works some sucks)

Follow